

ATCHAT Protocol Specification

A robust digital mode for amateur radio: SSTV—simple to operate, but able to carry image/file transfer with error correction (ARQ) plus a multi-station text net (NET) inside a single 2.7 kHz SSB channel. Inspired by KG-STV, EasyPal and HSModem.

Status. Software design + simulation. The modem is a real OFDM modulator/demodulator producing and decoding real audio, with real bit errors under real simulated noise. The “channel” is a TCP server carrying those samples; it is not yet wired to a sound card or an SDR. Version described: **0.1.1.**

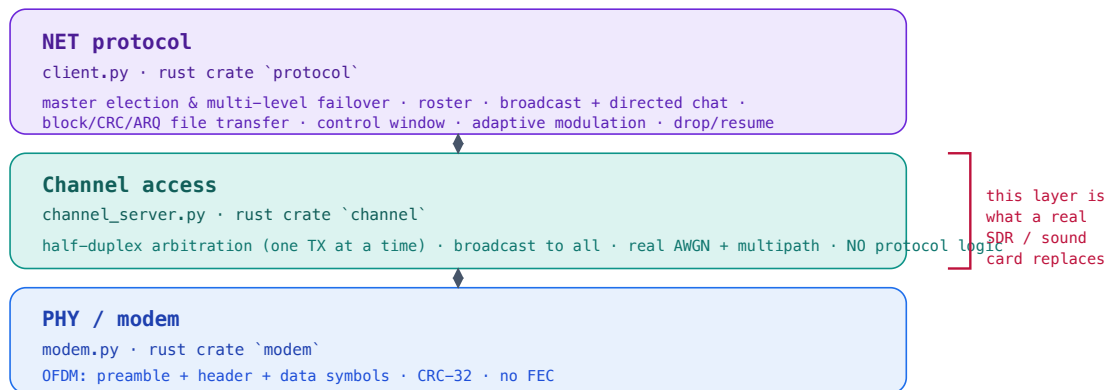
Contents

- | | |
|-------------------------------|---------------------------------|
| 1. Layering model | 6. Chat |
| 2. PHY layer — the OFDM modem | 7. Bulk transfer with ARQ |
| 3. Channel-access layer | 8. Timing & constant reference |
| 4. NET protocol frames | 9. Use cases & scenarios |
| 5. Session state machines | 10. Known limitations & roadmap |
| | 11. Component map |

1 Layering model

Three layers, deliberately separated. Moving to a real radio is expected to replace only the **channel-access** layer (TCP transport → sound card / SDR); the PHY and the NET protocol are unchanged.

Two implementations track each other bit-for-bit: the reference **Python** (`*.py` at the repo root) and a **Rust** port (`rust/`), cross-checked with vectors.



The layer split. Data flows down to modulate, up to demodulate; the channel layer holds no protocol state.

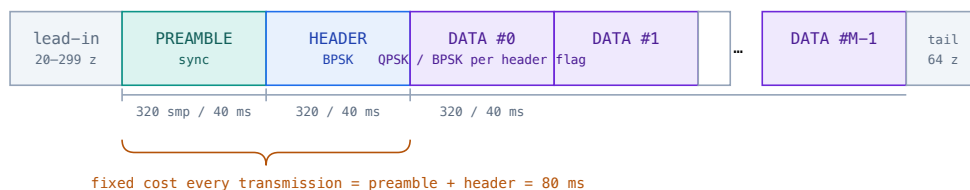
2 PHY layer — the OFDM modem

2.1 Parameters

Parameter	Symbol	Value	Notes
Sample rate	SAMPLE_RATE	8000 Hz	audio pass-band
FFT size	N	256	subcarrier spacing = $8000/256 = 31.25$ Hz
Cyclic prefix / guard	CP_LEN	64 samples	8 ms guard interval
OFDM symbol length	SYMBOL_LEN	320 samples	$N + CP_LEN \rightarrow 40$ ms on air per symbol
Data subcarriers	DATA_CARRIERS	bins 10...86 (77)	312.5 – 2687.5 Hz , inside 2.7 kHz
Preamble subcarriers	PREAMBLE_CARRIERS	even bins 2,4,...,126 (63)	Schmidl-Cox: even bins $\rightarrow$ two identical time-domain halves
Header bits	HEADER_BITS	17	16-bit length + 1-bit mode flag
Header repetition	HEADER_REPEAT	4	majority-voted at RX
Reference carrier	DATA_CARRIERS[0]	bin 10, fixed $1+0j$	carries no data; phase reference for differential coding
Info carriers	—	76	DATA_CARRIERS minus the reference
Integrity	—	CRC-32 (IEEE)	no FEC / LDPC / RS — correction is the NET layer's ARQ

The fixed preamble sequence is `numpy.random.RandomState(1234).choice([1,-1], 63)` — a constant both ends know; the Rust port embeds the same sequence.

2.2 On-air waveform structure

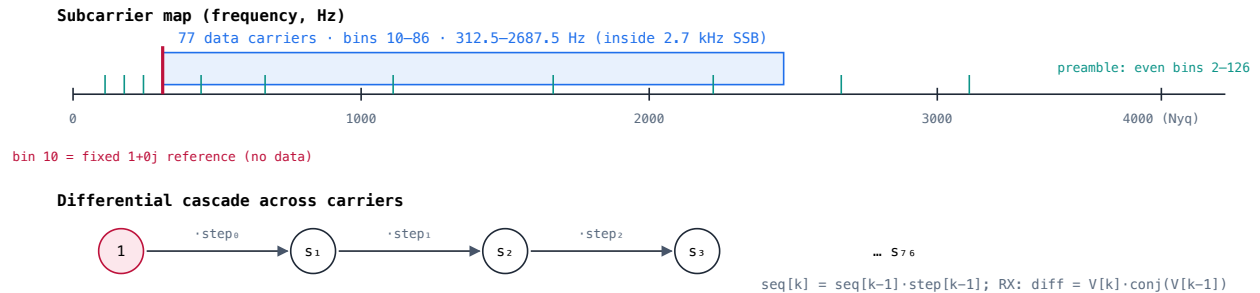


Every symbol: assign carriers $\{k: X[k]\} \rightarrow$ impose $X[N-k]=\text{conj}(X[k]) \rightarrow \text{real}(\text{IFFT}) \rightarrow \text{prepend CP}[-64:]$
Waveform peak-normalised to 0.7·32767, emitted as little-endian int16 PCM, base64 on the wire.

Waveform layout. A random lead-in forces true correlation-based sync; the 64-sample tail absorbs a few-sample sync slip.

2.3 Frequency-domain differential coding

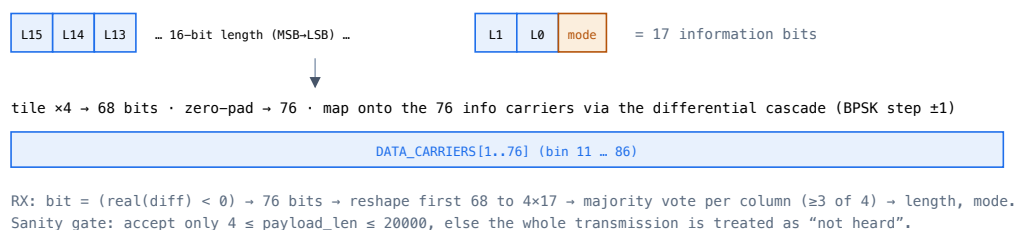
Both the header and the data symbols carry information as the **phase difference between adjacent subcarriers**, not as an absolute phase. A small sync error produces a phase ramp that is roughly linear in the carrier index k ; that ramp **cancels between neighbours**, leaving only a small constant residue. This buys noise robustness **without any channel estimation or equaliser** (classic differential OFDM).



Top: which FFT bins carry what. Bottom: information rides the running product of unit-magnitude “steps”, so the decoder only ever compares neighbours.

2.4 Header symbol (always BPSK)

Bits	Field	Meaning
0..15	payload_len	length in bytes of payload + 4-byte CRC , MSB first: bit $i = (payload_len \gg (15-i)) \& 1$
16	mode	1 = BPSK, 0 = QPSK — the modulation of the DATA symbols



The header is small, repeated and majority-voted — it must survive even when the data payload does not.

2.5 Data symbols

Mode	Bits / carrier	Bits / OFDM symbol (76 carriers)	Bytes / symbol	Data rate (payload)
QPSK (default)	2	152	19	19 B / 40 ms $\approx$ 475 B/s
BPSK (robust)	1	76	9.5	$\approx$ 237 B/s — $\sim 67\%$ slower, far more noise-tolerant

QPSK	$(b_0, b_1) \rightarrow \text{step}$	BPSK	$b \rightarrow \text{step}$
	$(0, 0) \rightarrow (1 + 1j)/\sqrt{2}$		$0 \rightarrow (1 + 0j)$
	$(0, 1) \rightarrow (1 - 1j)/\sqrt{2}$		$1 \rightarrow (-1 + 0j)$
	$(1, 0) \rightarrow (-1 + 1j)/\sqrt{2}$		
	$(1, 1) \rightarrow (-1 - 1j)/\sqrt{2}$		

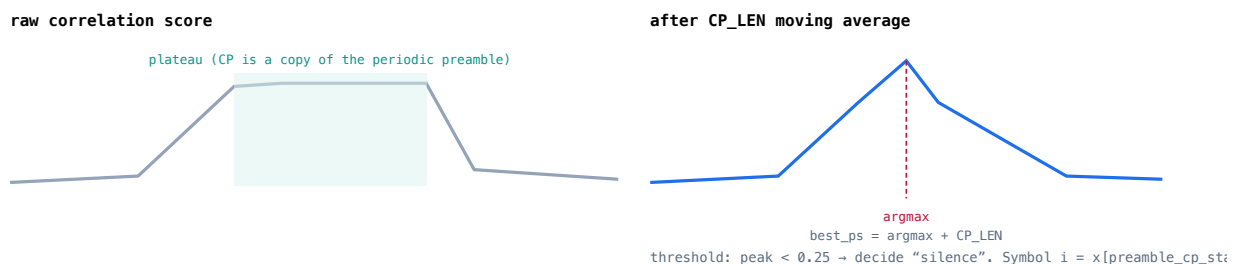
Bit-stream order is **MSB-first** over the payload bytes. For QPSK, carrier k carries bits $[2k, 2k+1]$. Decoding per differential value d : QPSK $\text{bit}_0 = \text{real}(d) < 0$, $\text{bit}_1 = \text{imag}(d) < 0$; BPSK $\text{bit} = \text{real}(d) < 0$. Concatenate all symbols' bits, truncate to $\text{payload_len} \cdot 8$, pack to bytes $\rightarrow$ `full`.

2.6 Payload framing & CRC

```
full = payload || CRC32(payload) as 4 bytes big-endian
```

RX splits `payload = full[:payload_len-4]` and **rejects the frame** (returns "nothing") if `crc32(payload) != crc_rcv`. There is no partial-frame recovery at the PHY — that is the ARQ layer's job.

2.7 Synchronisation (Schmidl-Cox style)



The single smoothed peak is robust; the raw plateau's edge is not. $i = 0$ preamble, 1 header, ≥ 2 data.

2.8 Time on air

```
bits_needed      = payload_len * 8           # payload_len already includes the CRC
bits_per_symbol  = 152 (QPSK) | 76 (BPSK)
data_symbols      = ceil(bits_needed / bits_per_symbol)
airtime           = (2 + data_symbols) * 320 / 8000  seconds  # = (2 + data_symbols) * 40 ms
```

Fixed cost per transmission: preamble + header = **80 ms**. A 220-byte file block, wrapped in its `BULK_BLOCK` JSON (~420 bytes on the wire), is ≈ 1.0 s on air at QPSK.

2.9 Measured performance (real, not modelled)

Condition	Result
Noiseless	100% (1 B ... 5000 B)
AWGN, SNR ≥ 18 dB	100%
AWGN, 14–16 dB	slight degradation
AWGN, 10–12 dB	sharp cliff for QPSK without FEC (expected COFDM behaviour)
AWGN, 10 dB	QPSK 0/20 vs BPSK 19/20 — the reason adaptive downshift exists
Multipath, delay ≤ 7 ms, echo ≤ -16 dB	solid (rides inside the 8 ms guard)
Multipath, delay > 8 ms or echo ≥ -10 dB	breaks on purpose — no channel estimator (§10)

3 Channel-access layer

3.1 Transport envelope (line-delimited JSON over TCP)

Each message is one UTF-8 JSON object followed by `\n`.

Client → channel

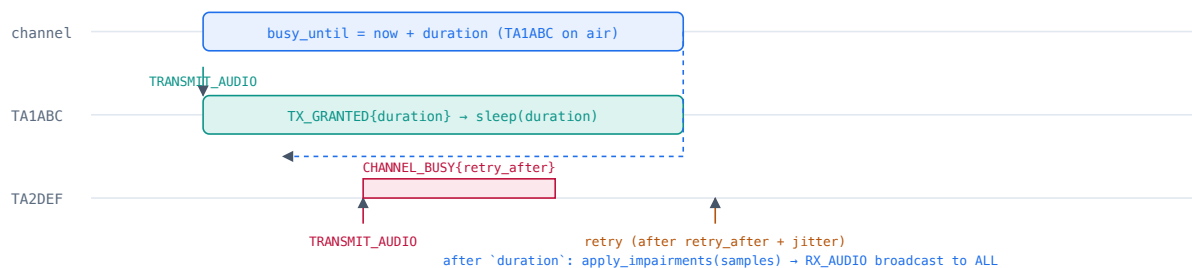
cmd	Fields	Meaning
HELLO	callsign	register on connect
TRANSMIT_AUDIO	audio_b64	base64 of little-endian int16 PCM — a full modem transmission

Channel → client

type	Fields	Meaning
TX_GRANTED	duration (s)	you hold the channel for <code>duration</code> ; the sender then sleeps that long ("keyed up")
CHANNEL_BUSY	retry_after (s)	someone else is transmitting; try again after <code>retry_after</code>
RX_AUDIO	audio_b64	a transmission delivered to every connected station (including the sender)

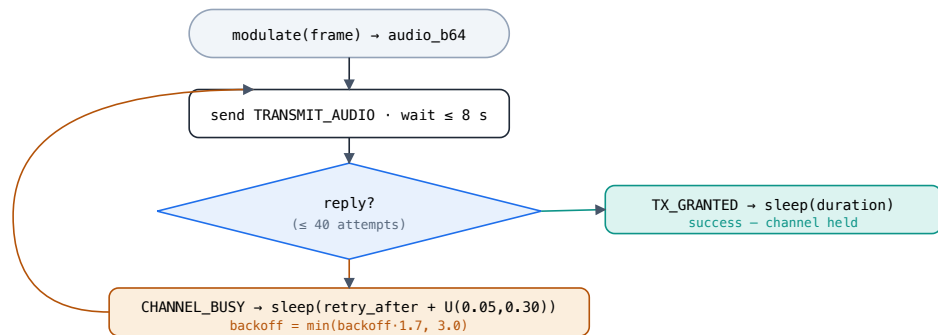
3.2 Half-duplex arbitration

The channel keeps a single `busy_until` timestamp. There is **no queue** and **no priority** — pure listen-before-talk; all back-off intelligence is on the client. Delivery is deliberately delayed by `duration` so airtime is physically modelled.



One transmitter at a time. The jittered client back-off (below) is what stops two stations colliding forever on the same `retry_after`.

3.3 Client-side LBT + exponential back-off



Only one frame at a time leaves a station (an async "PTT" lock). Give up after 40 attempts → "channel stayed busy".

3.4 Impairments (optional, for realism)

Applied in order to the int16 samples before broadcast: **(1) multipath echo** $x += \text{gain} \cdot \text{delay}(x, d)$ where $d = \text{delay_ms}/1000 \cdot 8000$ samples (meaningful up to the 8 ms guard); **(2) AWGN** with power $\text{mean}(x^2) / 10^{(\text{SNR_dB}/10)}$. CLI: `--snr`, `--multipath-delay-ms`, `--multipath-gain`.

4 NET protocol frames

The NET protocol lives **inside** the modulated payload: `frame` is a JSON object, `json.dumps`-ed, handed to `modem.modulate`, and recovered by `modem.demodulate` + `json.loads` on every receiver.

4.1 Common envelope

Field	Type	Notes
type	string	one of the types below
src	string	sender callsign
dst	string	"ALL" for broadcast, or a callsign for directed. Default "ALL"

A receiver acts on a frame when `dst ∈ {"ALL", own callsign}`. A station ignores its own `src`.

4.2 Frame catalogue

JOIN_REQUEST

`{type, src, dst:"ALL"}`. Announces arrival — sent on first connect and on every `/reconnect`. If a receiver has a half-finished *inbound* transfer from `src`, seeing this frame makes it re-issue the `BULK_STATUS` for the missing blocks.

BEACON

Field	Type	Meaning
src / dst	master callsign / "ALL"	
backup	string null	the assigned backup master (smallest active peer callsign)
roster	string[]	[master, ...all known callsigns]

Sent by the master every `BEACON_INTERVAL` (8 s), always in **BPSK**. It is the heartbeat, the roster sync, and the implicit `MASTER_CLAIM`.

CHAT

`{type, src, dst:"ALL"|callsign, text}`. Broadcast vs. directed. Sent in **BPSK** (robustness over speed for short text).

BULK_META

`{type, src, dst, transfer_id, filename, total_blocks, total_size}` — starts a file/image transfer. `transfer_id` is 8 hex chars; `total_blocks = ceil(total_size / 220)`.

BULK_BLOCK

Field	Type	Meaning
transfer_id	string	
seq	int	block index 0 ... total_blocks-1
data	string	base64 of ≤ BLOCK_SIZE (220) raw bytes
crc	uint32	crc32(raw_block) — per-block integrity

Sent in the transfer's current mode (QPSK, or BPSK after an adaptive downshift). A block whose `crc` fails on receipt is silently dropped; ARQ re-requests it.

BULK_END

`{type, src, dst, transfer_id}` — "that's all the blocks I have for now."

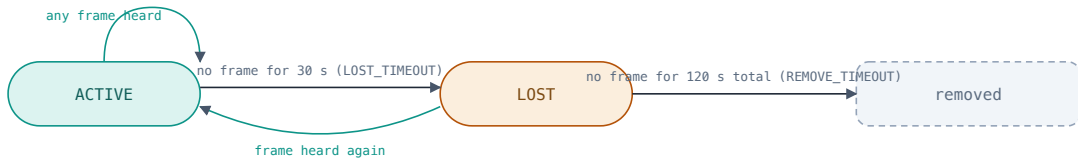
BULK_STATUS

`{type, src:receiver, dst:sender, transfer_id, missing:int[]}` — the receiver's ACK/NAK. `missing == []` ⇒ complete. Sent in **BPSK** from the end-of-transfer path; the drop/resume paths use the station's default mode.

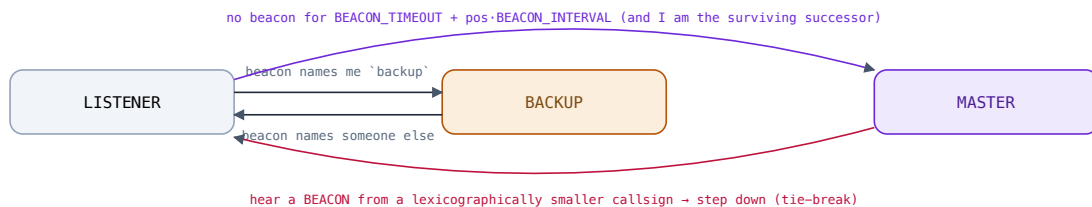
5 Session state machines

5.1 Roster

Every station keeps `roster: {callsign → {last_seen, status}}` locally, updated on every decoded frame from that callsign and from every BEACON's roster list.



5.2 Roles



On `/reconnect` a station forces itself to LISTENER and only sends JOIN_REQUEST — it never claims master while a beacon is alive.

5.3 Master election & multi-level failover

The old design chained through **one** assigned backup only. It now uses a **succession line** that every station derives identically from its own roster:

```
succession_line = sorted( {self} u {peers with status == ACTIVE} \ {presumed-dead master} )
pos              = index of self in that line

claim mastership when:  time_since_last_beacon > BEACON_TIMEOUT + pos * BEACON_INTERVAL
```

succession line (identical on every station)



Higher-priority survivors key up first. As a dead peer ahead ages to LOST it drops off the line and everyone below advances — so the chain continues arbitrarily deep, not just one backup.

Bootstrap (master == null): immediate takeover — the first station on an empty channel.

Tie-break for a simultaneous claim: the lexicographically smaller callsign wins; the other drops to LISTENER.

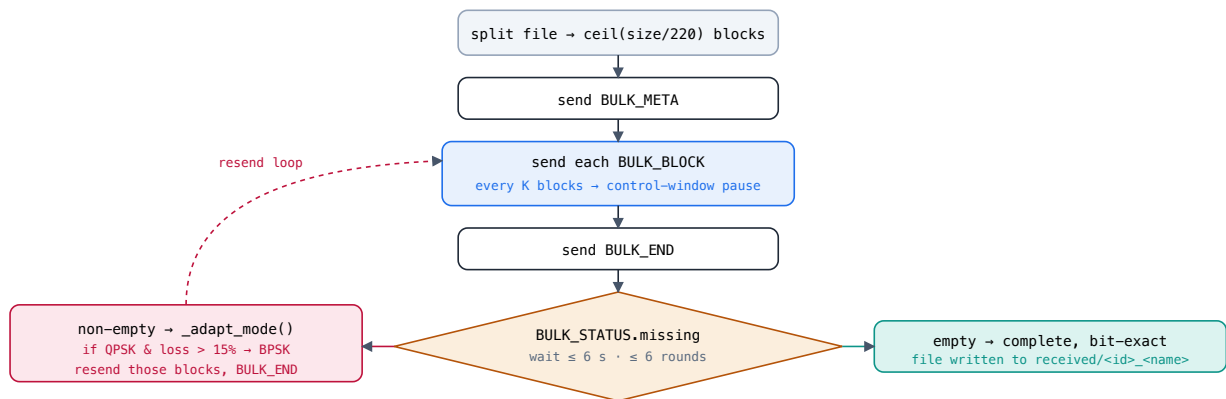
Implementation invariant (the deadlock rule). Any frame that must be *sent in response to a received frame* (BULK_STATUS after BULK_END , the resume BULK_STATUS after a JOIN_REQUEST , ...) is dispatched as

a **background task** — never `await`-ed on the single receive loop's call chain, because that loop is also what delivers the `TX_GRANTED` for that very send.

6 Chat

- **Broadcast:** `CHAT{dst:"ALL", text}`. Every station with `dst ∈ {"ALL", self}` displays it.
- **Directed / private:** `CHAT{dst:"TA2XYZ", text}`. Only `TA2XYZ` displays it; it still occupies the shared channel, it is just not *addressed* to anyone else.
- Sent in BPSK. Contends via the same LBT + back-off as any frame — nothing is dropped, it waits its turn, including waiting for a **control window** if a bulk transfer is in progress (§7.3).

7 Bulk transfer (image / file) with ARQ



Rounds exhausted → the transfer is **suspended**, not failed: it resumes automatically if the receiver reappears (§7.5).

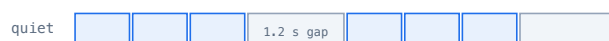
7.1 Receiver

1. `BULK_META` → create the inbound transfer, `received = {}`.
2. `BULK_BLOCK` → verify `crc32(data) == crc`; on match store `received[seq]`, on mismatch drop it.
3. `BULK_END` → `missing = [s for s in 0...total_blocks-1 if s not in received]`. Send `BULK_STATUS{missing}` (background task). If `missing == []`, write the file and mark complete.

7.2 Control window — keeping chat and beacons alive during a transfer

A back-to-back block stream would monopolise the half-duplex channel: chat, and even **beacons**, would starve, and stations would wrongly elect themselves master. So `_send_blocks` deliberately **pauses** to open the channel.

State	Pause every	Pause length
Quiet (baseline)	<code>CONTROL_WINDOW_EVERY</code> = 3 blocks	<code>CONTROL_WINDOW_PAUSE</code> = 1.2 s
Busy (contended)	1 block	1.5 s



“Contended” = a local chat/msg is queued, OR a peer sent CHAT / JOIN_REQUEST / BULK_STATUS within the last `CONTROL_CONTENTED_FOR` = 6 s. The window widens automatically and relaxes back on its own.

A competing station only knows roughly when a block ends (`retry_after`), not when the window opens – so it must be generous.

7.3 Adaptive modulation

No pilot-based per-subcarrier bit loading yet (needs a channel estimator — §10). Instead the ARQ round is the link-quality signal:

```
if mode == QPSK and (missing_this_round / blocks_sent_last_round) > 0.15:
    mode ← BPSK          # for the rest of this transfer; downshift only
```

The receiver needs no notification — every frame's **header carries its own mode flag**, so a transfer can switch mid-stream and still decode.

7.4 Drop / reconnect / resume

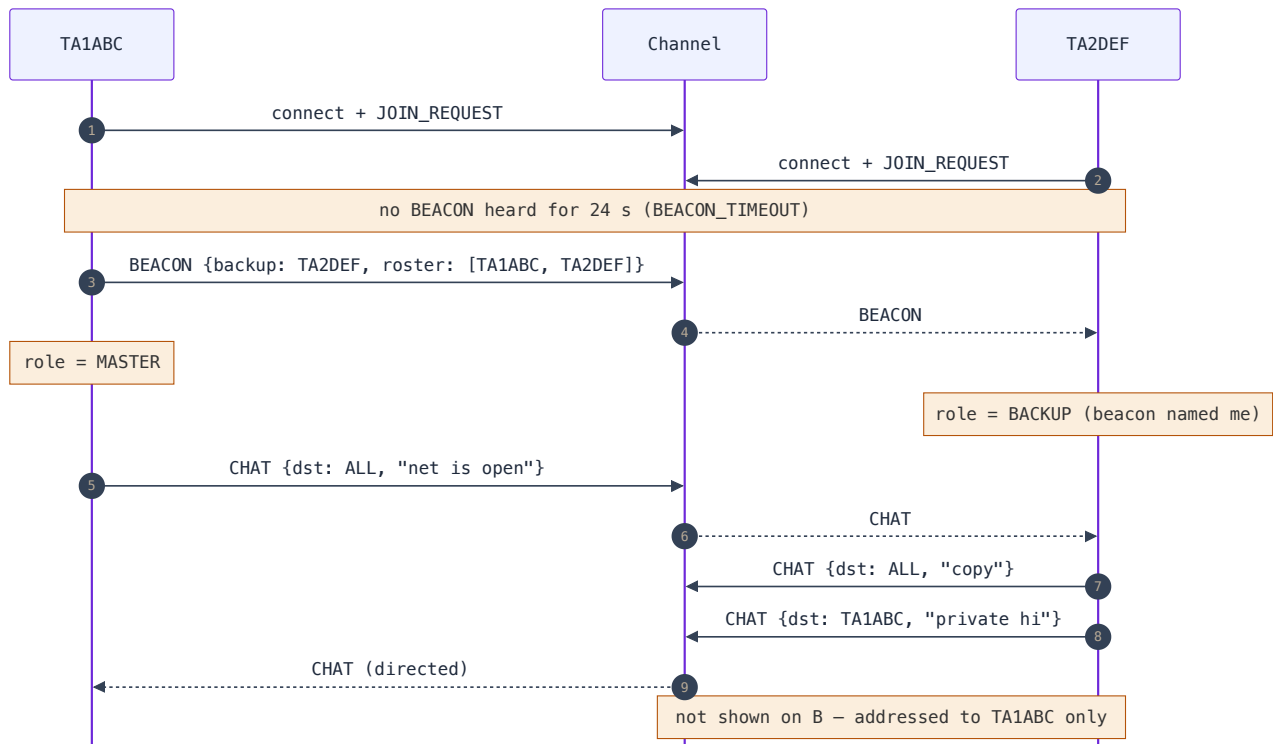
- `/drop` closes the TCP socket but keeps all state in RAM.
- `/reconnect` re-opens it, forces role LISTENER, sends `JOIN_REQUEST`, and re-requests missing blocks for every incomplete *inbound* transfer.
- The **other** side reacts to that `JOIN_REQUEST` (or to the station going ACTIVE again) by re-sending the blocks it still holds — even with no active ARQ loop, as a background task.
- Net effect: a station can vanish mid-transfer and, on return, the transfer **finishes only the missing part** — it never restarts.

8 Timing & constant reference

Constant	Value	Where
BEACON_INTERVAL	8 s	master beacon cadence
BEACON_TIMEOUT	24 s (3 × interval)	silence before a successor takes over
LOST_TIMEOUT	30 s	roster: ACTIVE → LOST
REMOVE_TIMEOUT	120 s	roster: drop entirely
BLOCK_SIZE	220 bytes	bulk block payload
CONTROL_WINDOW_EVERY / _PAUSE	3 blocks / 1.2 s	quiet control window
..._EVERY_BUSY / _PAUSE_BUSY	1 block / 1.5 s	contended control window
CONTROL_CONTENDED_FOR	6 s	how long “busy” persists after a peer control frame
ADAPT_DOWNSHIFT_FRAC	0.15	QPSK→BPSK loss threshold
ARQ rounds	6	<code>send_bulk</code>
BULK_STATUS wait	6 s	per ARQ round
<code>send_frame</code> retries	40	LBT attempts before giving up
back-off	0.2 s → ×1.7 → cap 3.0 s, + U(0.05, 0.30) jitter	per CHANNEL_BUSY
TX reply timeout	8 s	wait for TX_GRANTED / CHANNEL_BUSY
OFDM symbol	40 ms	320 samples @ 8 kHz
Preamble + header	80 ms	fixed per transmission

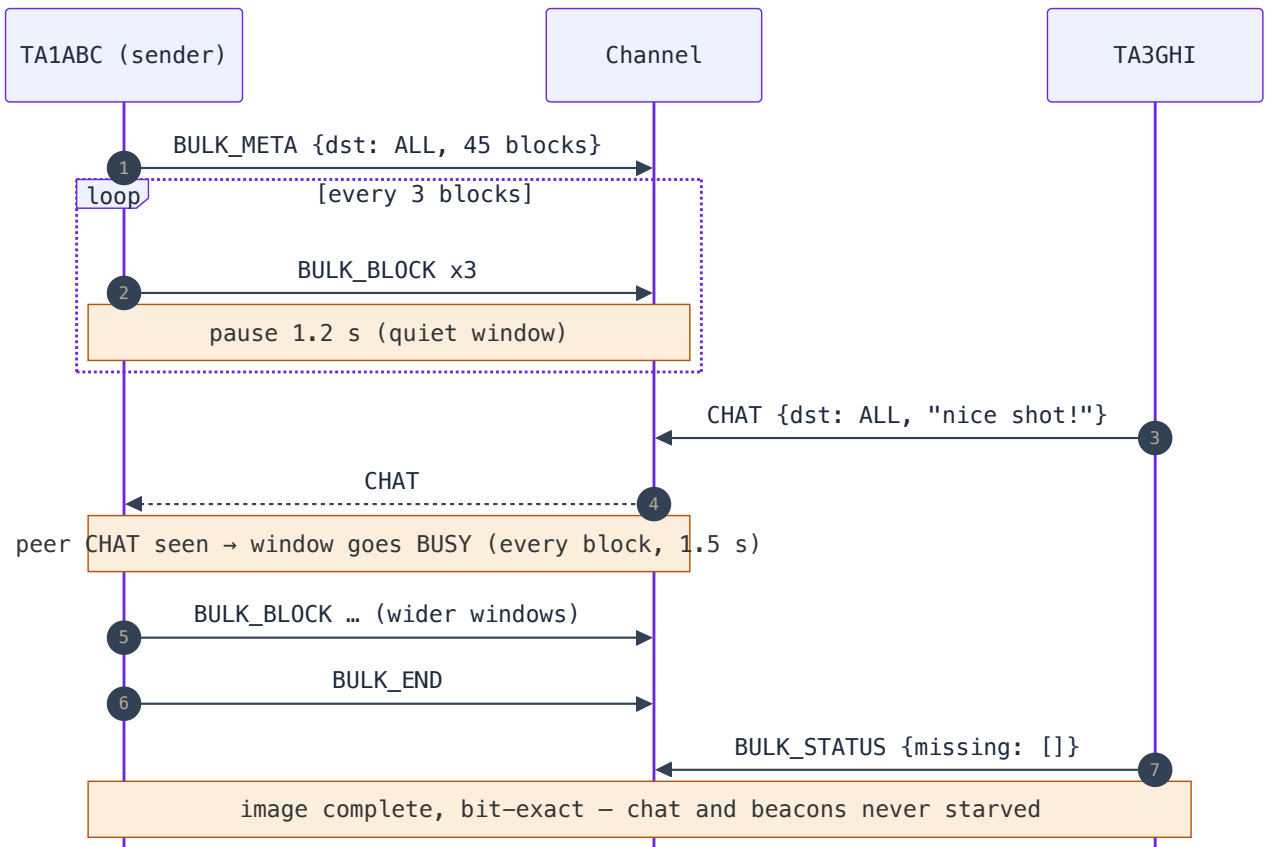
9 Use cases & scenarios

UC-1 — First two stations: election + chat

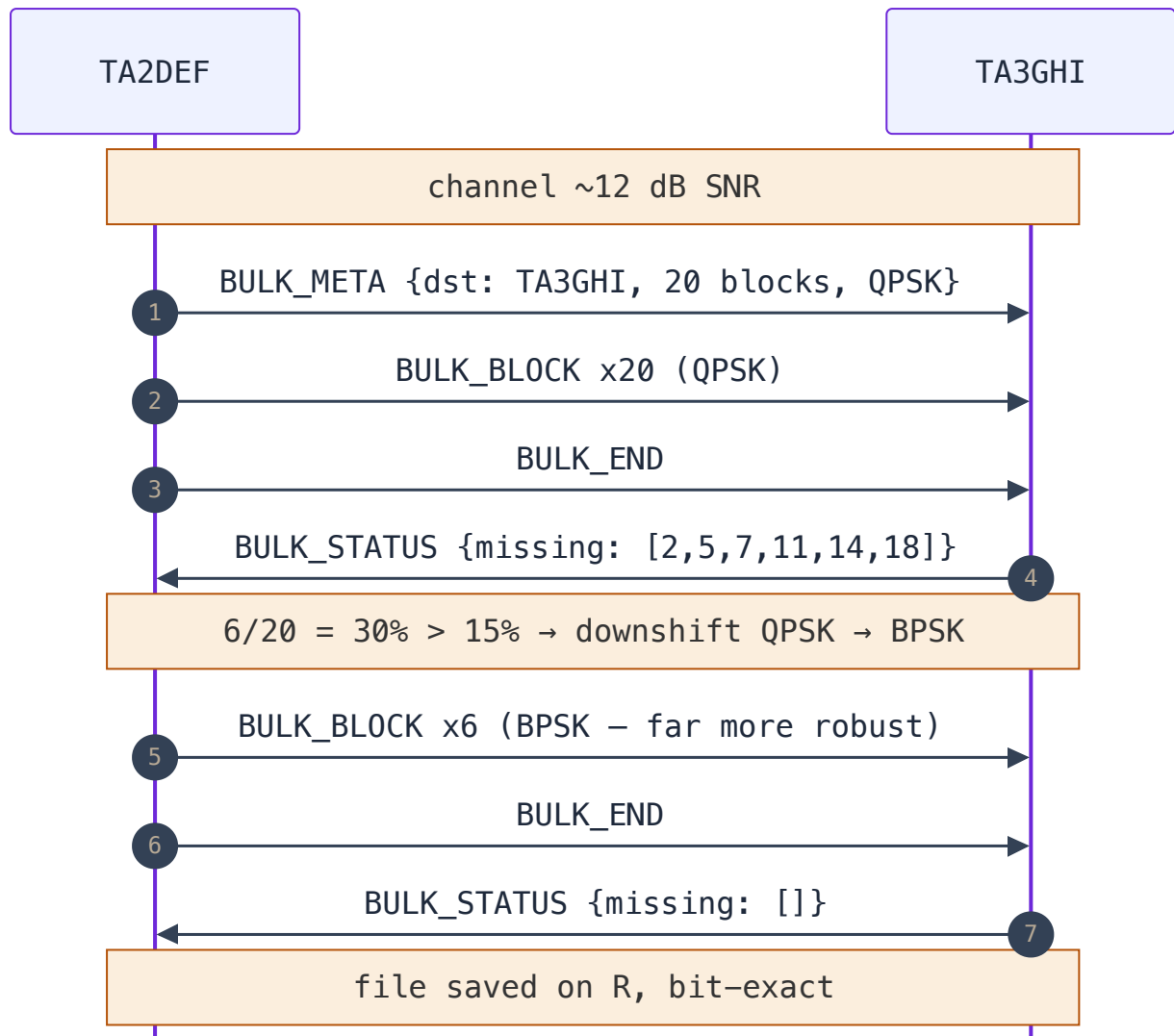


If both stations time out and beacon at once, the tie-break gives it to TA1ABC (smaller callsign); TA2DEF steps down and adopts BACKUP from the next beacon.

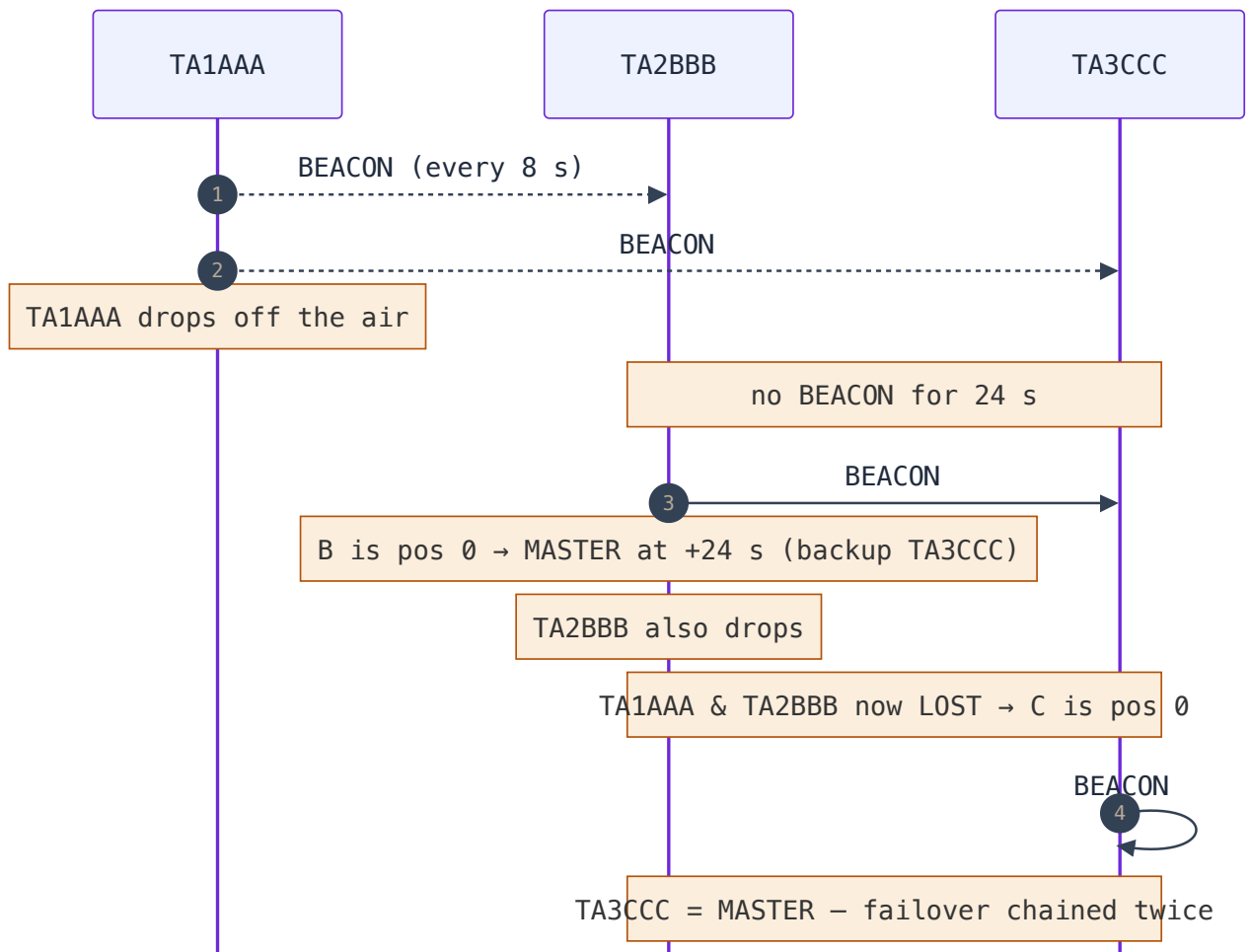
UC-2 — Group image while chatting



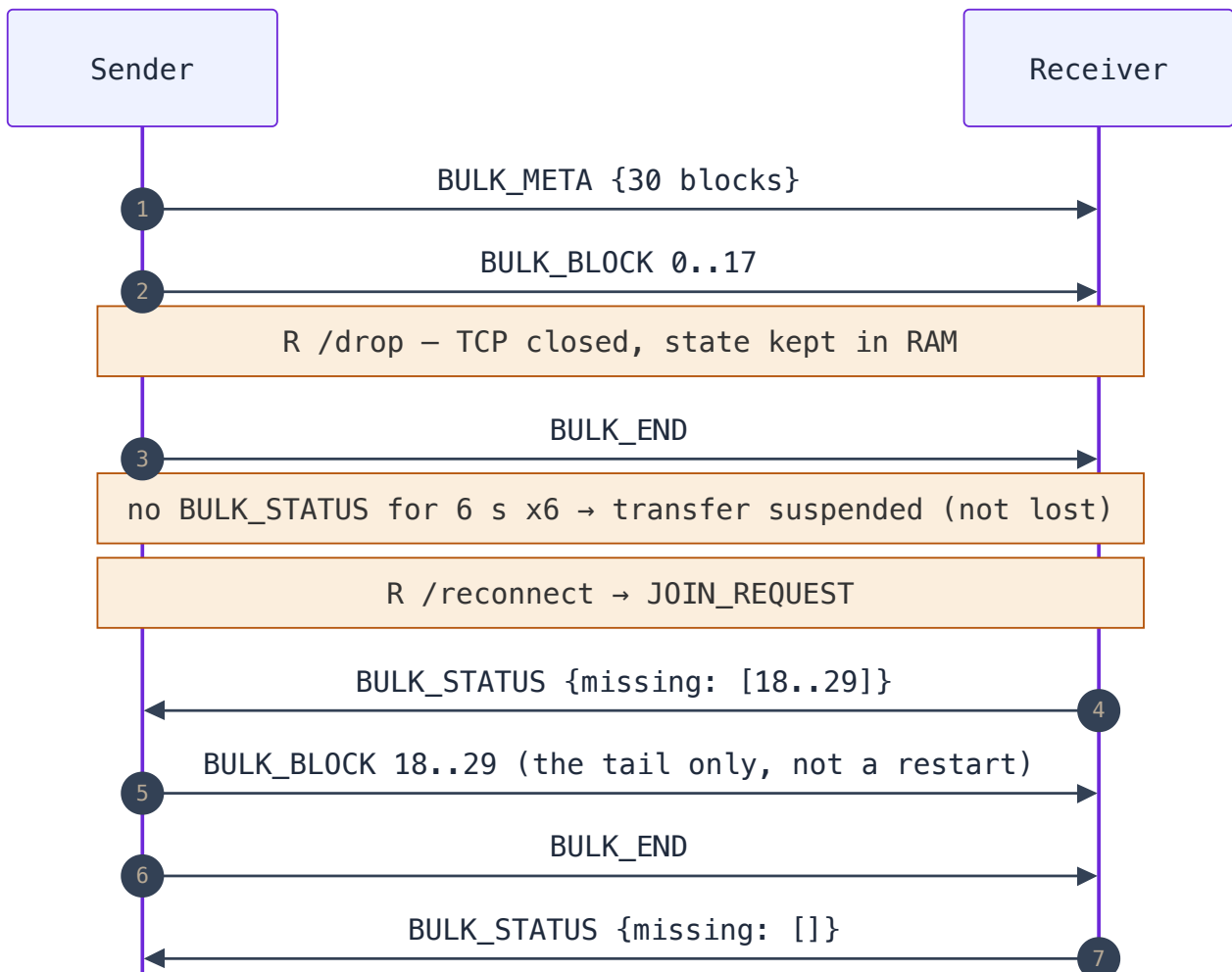
UC-3 — Private file under noise (ARQ + adaptive downshift)



UC-4 — Master failure → backup → chained failover



UC-5 — Station drops mid-transfer, resumes on return



10 Known limitations & roadmap

#	Limitation	Direction
1	Failover chained through one backup only	done — succession line (§5.3)
2	Multipath only verified for a <i>mild</i> echo end to end; a strong echo drives sustained loss into a <code>BULK_END</code> -loss ARQ stall	needs FEC + estimator
3	Full 4-station scenario verified in the Rust port; not yet with 4 real Python processes	—
4	Adaptive modulation is ARQ-loss-driven only (no per-subcarrier SNR bit loading)	needs #5
5	No channel estimation / equaliser	pilot-based estimation — next PHY step; unblocks real multipath
6	No LDPC/RS FEC — integrity is CRC-32, correction is ARQ only	a real next step
7	No sound-card I/O — audio still travels over TCP+base64	<code>sounddevice</code> / <code>cpal</code>
8	No SDR / RF integration	expected to touch only the channel-access layer
9	Control window is adaptive but its constants are fixed	could scale with pending traffic

11 Component map

File / crate	Role
<code>modem.py</code> / <code>rust/crates/modem</code>	OFDM modulator + demodulator (§2). Bit-exact between the two.
<code>channel_server.py</code> / <code>rust/crates/channel</code>	Channel physics + half-duplex arbitration + impairments (§3). No protocol logic.
<code>client.py</code> / <code>rust/crates/protocol</code>	The station: LBT, election/failover, roster, chat, bulk/ARQ, drop/resume (§4–7).
<code>monitor.py</code> / GUI Monitor tab	Passive listener — decodes the channel, never transmits, never in the roster.
<code>netproto.py</code> / <code>rust/crates/netproto</code>	Shared constants, CRC-32, line-JSON framing, frame definitions.
<code>rust/apps/atchat-gui</code>	Cross-platform egui app: Channel / Stations / NET / Monitor tabs + scope, spectrum, waterfall.